7.4.2 PLuMCoT

Finally, I propose here a meta-level generation control process conceived as a substantial generalization of APriCoT (section 5.6.2) in both task and structure. This process, dubbed Primed Lookahead using Multiple Chains of Thought (**PLuMCoT**), leverages the same primed free generation lookahead intermediary stage, but applies it at arbitrary generation timesteps and without limitation to MCQA-like pre-defined option sets. Designed as a drop-in replacement for the standard autoregressive generation process, PLuMCoT can be broken down into five (or six) distinct stages. How these fit together is illustrated in algorithm 1 and described below; its relationship to existing lookahead and decoding-time control methods is treated in section 7.4.2.8.

Algorithm 1 PLuMCoT generation loop

$ctx \leftarrow$ user input
$response \leftarrow \langle\,\rangle$
for each generation iteration **do**
    $b_{trig} \leftarrow \textsc{Trigger}(ctx)$ ▷ *Stage 1*
    if b_{trig} **then**
        $(\varphi, \lambda, \sigma, \omega) \leftarrow \textsc{Configure}(ctx)$ ▷ *Stage 1.5 (optional)*
        $\mathscr{V} \leftarrow forward(ctx)$ ▷ Main LLM inference
        $\mathscr{C} \leftarrow \textsc{Filter}_{\varphi}(\mathscr{V})$ ▷ *Stage 2*
        $\mathscr{S} \leftarrow \langle\,\rangle$
        for each candidate $c_i \in \mathscr{C}$ **do**
            $L_i \leftarrow \textsc{Lookahead}_{\lambda}(ctx, c_i)$ ▷ *Stage 3*
            $scr \leftarrow \textsc{Score}_{\sigma}(ctx \,\|\, c_i \,\|\, L_i)$ ▷ *Stage 4*
            $\mathscr{S} \leftarrow \mathscr{S} \,\|\, (scr, c_i)$
        end for
        $o \leftarrow \textsc{Select}_{\omega}(\mathscr{S})$ ▷ *Stage 5*
    else
        $o \leftarrow$ standard generation iteration
    end if
    $ctx \leftarrow ctx \,\|\, o$
    $response \leftarrow response \,\|\, o$
end for
return *response*

7.4.2.1 Stage 1: Trigger

The first stage, performed at the beginning of each autoregressive generation timestep, is *Trigger*. This stage determines, based on the current context and model state, whether this generation timestep should undergo PLuMCoT or call down to a single iteration of the standard (non-PLuMCoT) generation process. This stage is optional n principle, with omission being equivalent to always returning the value True so that PLuMCoT is to be used at every iteration. In practice, this is not recommended: PLuMCoT is, as the later stages will make clear, potentially quite computationally heavy, and this stage is intended to limit its application to iterations where the additional reasoning is expected to carry meaningful utility. A simple non-trivial example of a Trigger function that might perform well is an uncertainty threshold, activating PLuMCoT only

on generation timesteps where the model is sufficiently uncertain about the next token to be generated.

7.4.2.2 Stage 2: Filter

The second stage, *Filter*, reduces the tokens to be considered from the entire vocabulary, to a much smaller candidate pool, $\mathscr{C}$. This too is a necessary step to minimize the added computational overhead of stage 3. In most cases, this stage will involve an initial inference by the underlying primary model over the current context, or will reuse the same as originally generated for Trigger, as the next token probability will typically inform filtering. Many of the likely useful variations on this stage are already used in practice for standard generation processes. The common filtering functions, top-k and top-p, are the most natural choices here, though task-informed variations (e.g. a fixed set of label tokens for an MCQA task) may be contextually more effective and/or efficient.

7.4.2.3 Stage 3: Lookahead

Stage 3, *Lookahead*, is the defining stage of PLuMCoT and the stage most clearly inspired by APriCoT. This stage involves using the standard generation process (here serving as the base policy) to generate a continuation conditioned on the current context with each filtered candidate token appended to the end. For example, with starting context *ctx* and a candidate pool of three tokens, $\mathscr{C} = \langle x, y, z \rangle$, this stage generates three continuations $L_x = \text{generate}(ctx \,\|\, x)$ and similarly for L_y and L_z. The actual process used to generate these lookahead continuations can vary as much as the underlying generation functions provided by LLM inference libraries (e.g. the huggingface suite). The simplest possible version of Lookahead is a deterministic generation of up to a fixed number of new tokens. More complex and/or stochastic variations (e.g. top-p/k sampling and custom stopping criteria) are easily possible and may improve task efficacy at the expense of predictability and efficiency.

7.4.2.4 Stage 4: Score

In Stage 4, *Score*, each candidate is assigned a score that judges the quality of its simulated continuation, i.e. the context, the candidate token c_i, and the lookahead L_{c_i} it produced. Quality is left deliberately vaguely here, as the methods and attributes that determine it vary dramatically by task-specific and user-specific priorities. On the task-specific side, for example, a model engaging PLuMCoT for automated theorem proving may judge the continuation based on logical coherence and absence of fallacy, while one tasked with code generation may instead call down to a parser to verify syntactic validity. For more general-purpose applications, the simplest non-trivial scoring function is likely sequence perplexity (possibly conditional perplexity given the context and candidate token). A step up in complexity, and already exhibited in the APriCoT method, are

self-assessment methods where the model is prompted to judge its own output. Going further still, auxiliary learned scoring models, including other (likely smaller and possibly fine-tuned) LLMs, may prove worth their substantial additional computational cost. In many cases, particularly for general-purpose usage where numerous and possibly competing objectives (e.g. concept recall, alignment adherence, and explanatory clarity) must be jointly optimized, it may be necessary to compose and synthesize multiple otherwise distinct scoring methods.

7.4.2.5 Stage 5: Select

The final stage, *Select*, is comparatively straightforward and is tasked with deciding, based on the scores, which candidate token $c_i \in \mathscr{C}$ should be locked into the context for future generation timesteps. Only two general variations of this stage are predicted herein, but others may be conceivable and even beneficial. The easiest approach is a simple argmax or argmin to find the candidate token that led to the highest or lowest score after lookahead, depending on which direction is considered "preferred" by the chosen Score function. Where stochasticity is expected to be beneficial, this stage may instead compute the softmax over the scores and sample from the resulting probability distribution.

7.4.2.6 Optional Stage 1.5: Configure

Each of these stages is customizable, and where multiple variations are available, the optimal configuration is likely to be highly context-dependent. In principle, there is no requirement that the configuration remain constant across generation timesteps. This provides room for an optional but promising intermediary stage between Trigger and Filter, that herein will be called *Configure*. This stage would, given the context, select the combination Filter, Lookahead, Score, and Select from among the available variations that is expected to yield the best output. In this way, Configure may give a PLuMCoT workflow benefits similar to those of mixture-of-experts (MoE) models without the sparsely utilized additional trained parameters. Of the stages described here, this one appears to be the least anticipated by existing work: whereas the others have close analogues in the methods surveyed below in section 7.4.2.8, per-timestep reconfiguration of the search procedure itself does not, to my knowledge, have an established counterpart in the decoding-time control literature.

7.4.2.7 Hypothesized Benefits

PLuMCoT's structured lookahead is expected, at minimum, to provide significant reasoning improvements over a standard free generation procedure. This improvement comes from the increased search coverage over the space of possible responses, from the quasi-backtracking behavior that it provides, and, more formally, from the policy improvement property of rollout schemes discussed in section 7.4.2.8. Specifically for the

second of these, PLuMCoT allows a model to "experiment" with multiple possible continuations and determine whether a given token, if selected at this iteration, is likely to lead to undesirable or degenerate behavior. This is a weaker form of the explicit rewind mechanisms already present in the literature, obtained without the need to revisit committed tokens. Similarly, PLuMCoT may allow for the recognition of viable alternative continuations that seem immediately less preferred but may yield better downstream reasoning.

Even without any reasoning improvement, PLuMCoT carries a secong major expected benefit, sourced from the flexibility of the Score stage. To illustrate this, one can imagine a (relatively speaking) simple auxiliary boolean classifier mode trained to determine whether a given response violates a set of alignment protocols. Imposed as a hard requirement of the Score stage, for instance by returning a score of 0 for all violating lookaheads, this might prevent misaligned outputs much like standard methods such as reinforcement learning from human feedback (RLHF), but with at least three practical advantages. First, it would be expected to be more resilient to jailbreaking, since it does not rely on directly controlling the complexities of model behavior and instead reacts to that behavior when it is undesirable. Second, the resulting alignment would transfer across models by simply maintaining the same auxiliary model and Score definition, reducing the need for costly redundant alignment training. Finally, changing alignment protocols would be as "simple" as switching to an alternatively trained auxiliary model. Different users and different tasks may call for differing alignment needs, and this framework would allow for something akin to "hotswappable" alignment rather than the restrictive one-size-fits-all approach typified by standard RLHF.

7.4.2.8 Relation to Existing Work

While PLuMCoT was originally conceived purely as a generalization of APriCOT, it is best understood as a generalization and unification of several existing lines of work. The most similar identified pre-existing method is CoT-decoding (Wang and Zhou, 2024), which branches over the top-k alternatives at the first decoding step, continues each greedily, and selects among the resulting paths by the model's confidence in its final answer. In the present terms, this is a PLuMCoT instance with a top-k Filter, a deterministic Lookahead, and a self-report confidence-based Score. Notably, that work explicitly identifies branching at arbitrary timesteps, and searching for the best path during decoding, as unaddressed future work, citing computational cost and the difficulty of reliably identifying the best token as the obstacles. These are exactly the concerns that the Trigger and Score stages are respectively designed to address. Structurally similar is NeuroLogic A*-esque decoding (Lu et al., 2022), which similarly pairs candidate filtering with lookahead-based estimates of future cost, drawn from perplexity or constraint satisfaction, and which was offered as a drop-in replacement for beam search and top-k sampling. PLuMCoT differs in leaving Score unconstrained rather than binding it to a lexical constraint objective.

The alignment argument advanced in the preceding subsection also has direct precedent in the decoding-time alignment literature, which steers a frozen base model at inference rather than updating its weights. RAIN (Li et al., 2024) couples self-evaluation with a rewind mechanism to align frozen models with no additional data, training, or gradient computation, while controlled decoding (Mudgal et al., 2024) trains a prefix scorer as a value function for a given reward. The latter demonstrates that scorers for multiple rewards can effectively compose at inference time, and that their benefit transfers to an unseen base model without additional finetuning. As such, the hypothesized alignment benefits are, at current, well founded in the existing literature.

Beyond its resemblance to specific decoding methods, PLuMCoT fits within the space of classical combinatorial search methods. Taking the partially generated response as the state and the LLM as the successor function, the PLuMCoT loop is a greedy lookahead search over response space in which candidates are evaluated by lookahead simulation rather than by a static heuristic. More precisely, it is a slightly degenerate instance of filtered beam search (Ow and Morton, 1988) with the beam width fixed at one. Filtered beam search is defined by a two-tier evaluation in which a cheap filtering priority reduces the successors of a node to a small candidate set and a more expensive evaluation is applied only to the survivors, exactly matching the division of labor between the Filter and Score stages described above. The Lookahead stage, meanwhile, plays a role similar to that of rollouts in Monte Carlo tree search (Kocsis and Szepesvári, 2006; Browne et al., 2012), estimating the value of a candidate by simulating forward from it under a base policy. The resemblance is only partial, as PLuMCoT retains no tree across timesteps, accumulates no visit statistics, and applies no selection policy to balance exploration against exploitation, so it is better described as borrowing the simulation step of MCTS than as implementing MCTS itself. Each of these omissions is also an available extension, as are widening the beam past one, revisiting committed tokens under a restart policy, and varying the lookahead horizon with the difficulty of the current timestep.

Finally, PLuMCoT also finds itself under the umbrella of model predictive control by virtue of being, in structure, an instance of receding-horizon control. At each step, candidate actions are simulated forward over a finite horizon, scored against a cost, and only the first action is committed before the process repeats. More specifically, it is a truncated rollout scheme in the sense of Bertsekas (2022), with the standard autoregressive decoder serving as the base policy and Score supplying the terminal cost approximation. In the general case, rollout over a base policy carries a policy improvement guarantee under suitable conditions on that base policy (Bertsekas et al., 1997), which offers a stronger basis for the reasoning benefits hypothesized above than search coverage alone. It should be noted that PLuMCoT cannot claim such a guarantee unconditionally, as it depends on the details of the Score implementation selected from the unconstrained set of all possible variations. The uncertainty-gated Trigger, in turn, bears some similarity to the adaptive computation literature,

such as Schuster et al. (2022), who implement confidence-gated early-exit mechanisms in LLM inference. While unexplored here, frameworks of that kind can cleanly co-exist with PLuMCoT by relegating standard inference to timesteps where the model is neither highly certain nor too uncertain.

7.4.2.9 A Final (Hypothesized) Human Similarity

A final expected effect of the PLuMCoT framework is of particular interest to the research program described in this dissertation. Many of the stages may be designed such that generation complexity increases with increased uncertainty. Trigger may prevent Lookahead from executing at all only under except under substantial uncertainty. Filter, in many plausible variations, will admit more candidate tokens under high uncertainty, making the Lookahead and Score stages proportionally more costly. Lookahead could conceivably benefit from rolling out longer when uncertainty is high. In all cases, the expected result is that uncertain generation timesteps take longer to complete. This has strong potential to manifest as an uncertainty-moderated graded response time differential, possibly allowing direct comparison between human and LLM response times in future human-AI similarity studies, rather than the loosely justified token-probability proxy relied on in many of the studies described herein.